DS 501 · INTRODUCTION TO DATA SCIENCE

Week 03

Data Sources, Formats and Acquisition

Where data comes from, how it is stored and described, and how to collect it with Python from files, APIs and web pages.

THIS WEEK

- **3-1** Data Sources and Collection — 45 minutes
- **3-2** Formats, Metadata and Tidy Data — 45 minutes
- **3-3** Acquiring Data with Python — 45 minutes
- **P3** Practice: files, APIs, scraping, tidy data — 60 minutes · Python notebook

International Balkan University · [Instructor name] · [Semester]

Recap of Week 2

Three ideas from the data science methodology that this week builds on:

The question decides the data

Data requirements follow from the business question and the analytic approach, not from what happens to be available.

Understand before you model

Data understanding checks content, quality and gaps early, and often sends you back to collect more or different data.

The process is a cycle

CRISP-DM loops back. Preparing data is usually one of the longest stages, so it pays to collect and store it well.

This week fills in the **data collection** stage: where data comes from, what form it takes and how to get it with Python.

DS 501 · WEEK 3 · LESSON

3-1

Data Sources and Collection

Lesson 1 of 3

In this lesson

01 Kinds of data source: internal or external, primary or secondary

02 How databases, files, APIs, web pages and streams deliver data

03 Legal, ethical and sampling limits on collecting data

04 Judging whether a source is fit for your question

45 minutes

Builds on week 2: data requirements

Objectives

By the end of this lesson, you will be able to:

01 **Distinguish** internal and external, primary and secondary data sources

Checked in: Quiz 3 · question 1

02 **Describe** how databases, files, APIs, web pages and streams deliver data

Checked in: Quiz 3 · questions 2–3

03 **Explain** the legal, ethical and sampling limits on collecting data

Checked in: Quiz 3 · question 4

04 **Evaluate** whether a data source is fit for a given question

Checked in: Quiz 3 · questions 5 and 16

Look for the objective chip in the top-right corner of each slide: it shows which goal that slide serves.

Objective 01

Case: Where Will the Data Come From?

THE SITUATION

A bakery chain with several shops in Skopje throws away too much bread at the end of each day. The owner wants to predict tomorrow's demand for each product in each shop.

> “We have our till data. What else do we need, and how do we get it?”

The till system records every sale with the shop, the product, the time and the price.

YOUR TASK · PAIRS · 5 MIN

1. List every data source that could help to predict demand.
2. Sort your list: inside the company, or outside it?
3. For two outside sources, say how you would obtain the data.

Two Ways to Classify a Data Source

Ask two questions: who produces the data, and for whose question was it collected?

	Primary: collected for your question	Secondary: collected by others, reused
Internal	A customer survey your company runs; an A/B test on your own website	Sales, CRM and HR records kept to run the business
External	A survey a market-research firm runs for you, to your design	State Statistical Office tables; weather archives; World Bank indicators

In practice

Most projects reuse secondary data, because it is cheaper and faster than collecting new data.

Read the right column first. Secondary data was designed for someone else's question.

Where Data Comes From

Source	Origin	Typical format	Watch out for
Sales, orders and payment systems	Internal	Relational database	Access rights; definitions
CRM, ERP and HR applications	Internal	Database extracts	Personal data
Web logs, app events, sensors	Internal	JSON, CSV, streams	Volume; time zones
Open government data	External	CSV, XLSX, APIs	Definitions change by year
Commercial and partner data	External	Files, APIs	Cost; licence; matching keys
Research datasets and the web	External	CSV, HTML, JSON	Unknown collection method

In practice

Start local: the State Statistical Office and the national open data portal publish free tables and files.

Note: most projects join an internal and an external source. A shared key decides whether that is possible.

Objective 02

Databases, Flat Files and Spreadsheets

Relational databases

Structured data in related tables, queried with **SQL**.

Most internal operational data lives here.

Analysts usually get read access or an extract.

Flat files (CSV, TSV)

Plain text: one record per line, values split by a **delimiter**.

One file holds one table.

The most common format for exchanging data.

Spreadsheets (XLSX)

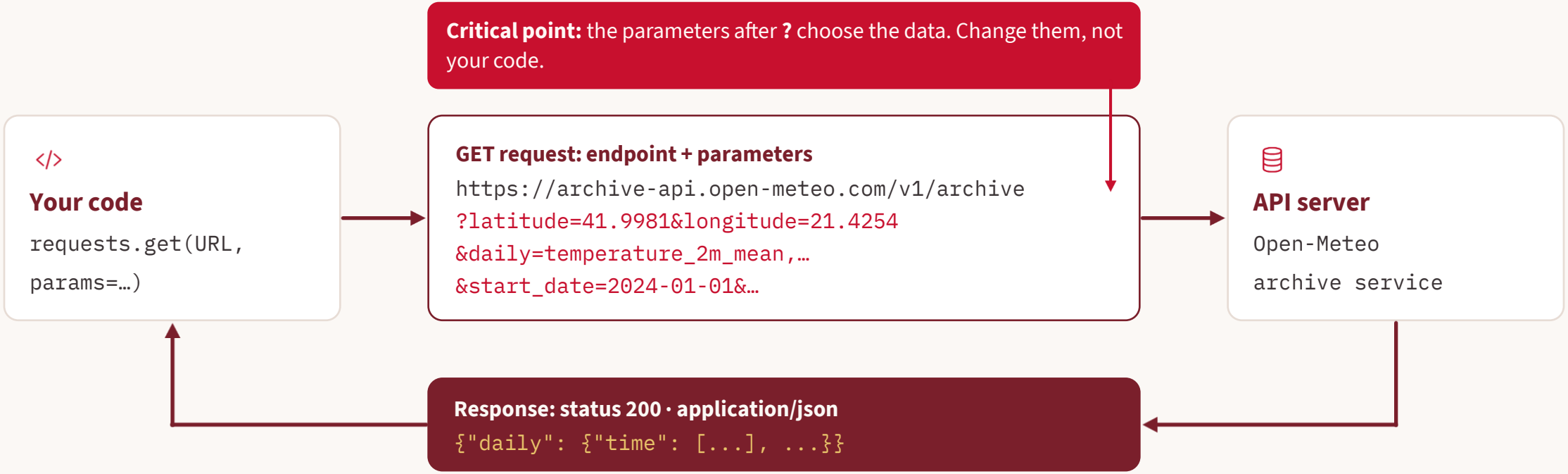
A workbook can hold several **sheets**, one table each.

Formulas, colours and merged cells mix with data.

Easy to edit by hand, so easy to break.

⚠ **Common mistake:** treating a spreadsheet as a database. Merged cells, totals rows and colour codes are not data and must be removed first.

How an API Delivers Data



- An **API** is a published door into another system: you ask with a URL and parameters, it answers with data.
- Practice 3 sends exactly this request, for daily Skopje weather in 2024.

Objective 02

Batch Data Versus Streaming Data

Aspect	Batch	Streaming
How data arrives	In chunks, at scheduled times	Continuously, record by record
Delay (latency)	Minutes to days	Seconds or less
Typical source	Nightly export, monthly file	Sensors, clicks, card payments
Typical use	Reports, model training	Alerts, fraud checks, live dashboards
Example tools	SQL extracts, scheduled scripts	Apache Kafka, Spark Structured Streaming

Why? Choose streaming only when the decision must be made within seconds; most projects use batch data.

Key point: the speed of the **decision**, not the speed of the data, decides between batch and streaming.

Web Scraping

- **Web scraping** is the automatic extraction of data from web pages written for people, not for programs.
- A scraper downloads a page, finds the elements that hold the data and saves them as a table.
- Typical uses: price comparison, job adverts, public notices, and text for training models.
- Scrapers are **fragile**: when a site changes its layout, they break without warning.

Common tools

- pandas read_html for simple tables
- BeautifulSoup to find tags and text
- Scrapy to crawl many pages
- Selenium or Playwright for JavaScript pages

Key point: prefer an official API or a published download; scrape only when neither exists.

Can You Use Data Just Because You Can See It?

- Visible in a browser does not mean **free to reuse**: read the site's terms of service first.
- **robots.txt** says which parts of a site automated agents should not visit; respect it.
- Copyright and the EU **database right** can protect collections of facts, not only texts.
- Names, photos and posts about people are **personal data** under the GDPR, even when public.

Good practice

- Use an official API if one exists
- Pause between requests
- Identify your scraper
- Collect only the fields you need

⚠ **Common mistake:** assuming that public data is open data. Open data comes with a licence that allows reuse.

Collecting Your Own Data: Three Biases

When no source answers your question, you collect primary data: a survey, an experiment, observation or sensors.

Selection bias

The sample leaves out part of the population.

Example: an online survey misses people without internet access.

Non-response bias

People who reply differ from those who do not.

Example: only very unhappy customers answer a satisfaction survey.

Measurement bias

The question or instrument pushes values one way.

Example: “How much did you enjoy the course?” assumes enjoyment.

Key point: plan the sample, the questions and consent **before** you collect a single answer.

Objective 04

Secondary data was collected to answer someone else's question.

Its definitions, coverage and timing reflect that original purpose, not yours. Before you reuse it, find out who collected it, why, how and when.

Rule of thumb: read the documentation before you read the data.

Objective 04

Evaluating a Data Source

Criterion	Question to ask
Relevance	Does it measure what the question needs, at the right unit of analysis?
Quality	Is it complete, accurate, consistent and documented?
Timeliness	How recent is it, and how often is it updated?
Representativeness	Does it cover the population the decision applies to?
Access and cost	Can you get it in time and within budget?
Licence and legality	May you use it for this purpose? Does it hold personal data?
Joinability	Does it share a key (date, region, ID) with your other sources?

In practice

Use these seven questions for every source in your week 3 project milestone.

Critical point: joinability is the criterion most often forgotten. No shared key, no combined analysis.

Exercise: Which Source, and How?

FIVE DATA NEEDS

1. Last year's sales per shop, from the company's own system.
2. Hourly temperature forecasts for Skopje.
3. Customer satisfaction with a new delivery service.
4. Prices that competitors show on their websites.
5. Card payments, to block fraud within one second.

“Find the cheapest legal way to get each one.”

YOUR TASK · PAIRS · 5 MIN

1. Choose a channel for each need: database extract, API, scraping, stream or survey.
2. Classify each source: internal or external, primary or secondary?
3. Which need raises a legal question, and what would you check first?

Objective 01

Check Your Understanding

A researcher studies regional unemployment using tables published by the State Statistical Office from its Labour Force Survey. How is this source best classified?

A Internal and primary, because the researcher chose the tables.

B External and secondary: collected by another body, for its own purpose.

C External and primary, because a survey is always primary data.

D Internal and secondary, because the data is published officially.

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Data source
Any origin from which data is obtained for analysis.

Primary data
Data collected by you, or for you, to answer your question.

API
A published interface that returns data in response to a request.

Data stream
A continuous flow of time-stamped records, processed as they arrive.

Internal / external source
Produced by the organisation itself, or by someone else.

Secondary data
Data collected by others for another purpose, then reused.

Web scraping
Automatic extraction of data from web pages written for people.

Sampling bias
A sample that differs systematically from the population of interest.

Summary

In this lesson, you learned how to:

- **O1** Distinguish internal and external, primary and secondary data sources
- **O2** Describe how databases, files, APIs, web pages and streams deliver data
- **O3** Explain the legal, ethical and sampling limits on collecting data
- **O4** Evaluate whether a data source is fit for a given question

Key takeaway: every source answers someone's question first; check its purpose, rights and fit before you rely on it.

Next lesson

3-2 Formats, Metadata and Tidy Data

DS 501 · WEEK 3 · LESSON

3-2

Formats, Metadata and Tidy Data

Lesson 2 of 3

In this lesson

01 Text and binary file formats, and what each keeps

02 Pitfalls of reading text files: encodings, decimals, dates

03 Metadata, data dictionaries and data catalogues

04 Tidy data: one variable per column, one observation per row

45 minutes

Builds on lessons 1-2 and 3-1

Objectives

By the end of this lesson, you will be able to:

01 **Compare** text and binary, row-based and columnar file formats

Checked in: Quiz 3 · question 6

02 **Identify** encoding, delimiter, decimal, date and missing-value pitfalls in text files

Checked in: Quiz 3 · question 7

03 **Distinguish** technical, process and business metadata, and write a data dictionary

Checked in: Quiz 3 · question 8

04 **Explain** the three tidy data principles and the common messy-data problems

Checked in: Quiz 3 · question 9

05 **Apply** melt to reshape a wide table into tidy, long form

Checked in: Quiz 3 · question 10 and Practice 3

Look for the objective chip in the top-right corner of each slide: it shows which goal that slide serves.

Objective 04

Case: The Spreadsheet from Finance

THE SITUATION

Finance sends you an Excel workbook of monthly sales by region for the last two years.

It has one column per month, merged header cells, a totals row after each region, and amounts written as “1.234,50”.

“Here is the file. Can you have the analysis ready by tomorrow?”

YOUR TASK · PAIRS · 5 MIN

1. What would go wrong if you loaded this file into pandas as it is?
2. What should one row of a clean table represent?
3. Which problems would you fix in code, and which would you ask Finance about?

Objective 01

What Each File Format Keeps

Format	Layout	Keeps data types?	Typical use
CSV / TSV	Text, one row per line	No: everything is text	Exchanging flat tables
XLSX	Zipped workbook of sheets	Partly: cell types	Business reporting
JSON	Text, nested key–value pairs	Partly: number, text, true/false	Web APIs, documents
XML	Text, nested tags	Only with a schema (XSD)	E-invoices, older systems
Parquet	Binary, columnar, compressed	Yes: schema in the file	Analytics, data lakes, Spark
Avro	Binary, row-based	Yes: schema in the file	Streaming messages (Kafka)

Critical point: CSV keeps no data types; Parquet does.

⚠ **Common mistake:** saving every intermediate result as CSV, so the column types are lost and guessed again at each step.

Objective 01

Row Storage Versus Columnar Storage

- A **row** format stores one record after another: good for writing and reading whole records.
- A **columnar** format stores each column together: good for reading a few columns of many rows.

Objective 02

Pitfalls When Reading Text Files

Pitfall	Example	pandas setting
Encoding	“Скопје” or “Kërçovë” shown as garbled symbols	encoding="utf-8" (or "cp1251")
Delimiter	Excel in a European locale exports with “;”	sep=";"
Decimals	“1.234,50” means one thousand two hundred…	decimal=",", thousands="."
Dates	03/04/2025: 3 April or 4 March?	parse_dates=[…], dayfirst=True
Missing codes	Blank, “-”, “n/a” or 9999 all mean missing	na_values=["-", "n/a", 9999]

Note: a number column read as text (object) almost always points to row 3 or row 5.

In practice

Prefer UTF-8, ISO 8601 dates (2025-04-03) and one documented missing-value code in every file you publish.

Three Kinds of Metadata

Metadata is data that provides information about other data.

Technical

Describes **structure**: tables, columns, data types, row counts.

Example: pm25 is a decimal number; the table has 366 rows.

Process

Describes how data was **produced and moved**: jobs, times, sources.

Example: loaded from the API on [date] by script X.

Business

Describes **meaning** in business terms: definitions, owners, rules.

Example: “active customer” = a purchase in the last 90 days.

In practice

In a relational database, technical metadata lives in the **system catalogue**; you will meet it again in week 4.

Objective 03

Metadata for the Practice 3 Weather Data

Type	Example for the Skopje daily weather table
Technical	Columns time (date), temperature_2m_mean (number), precipitation_sum (number); 366 rows for 2024
Technical	Units °C and mm; time zone Europe/Skopje; encoding UTF-8
Process	Requested from the Open-Meteo archive API on [date] by the Practice 3 notebook
Process	Monthly table derived by resampling: mean temperature, total precipitation
Business	Daily mean air temperature 2 m above ground in Skopje; a driver of demand
Business	Licence and citation terms of the data provider

Why? Process metadata is what lets someone else rebuild your file.

In practice

The notebook prints which source it used. If the API is unreachable, it uses a **synthetic** sample and labels it.

A Data Dictionary

A data dictionary documents every column of one table. Example: the quiz table from Practice 3, in long form.

Column	Type	Values / unit	Description	Missing
student	text	S01–S04	Anonymised student ID	0 of 12
programme	category	MSc DS, MSc CS, MBA	Study programme	0 of 12
week	integer	1–3	Week of the quiz	0 of 12
score	decimal	points	Quiz score	1 of 12

In practice

A **data catalogue** is a searchable inventory of many such dictionaries across an organisation.

Note: record missing values here, so nobody mistakes them for zeros.

Tidy Data

- Wickham (2014) defines **tidy data** as a standard way to match a dataset's meaning to its structure.
- Each **variable** is a column, each **observation** is a row, and each type of observational unit is a table.
- pandas, seaborn, scikit-learn and SQL all expect this shape, so tidy data needs no special code.
- Tidy data is Codd's third normal form, stated in the language of statistics.

Three rules

- Variable → column
- Observation → row
- Observational unit → table

Key point: decide what one row represents before you reshape anything.

Objective 04

Decide what one row means before you clean or reshape anything.

Tidy is defined relative to the question. Daily readings per station are tidy for a pollution model; monthly totals per city are tidy for a policy report.

- Rule of thumb: write “one row = one …” at the top of every notebook.

Five Common Problems of Messy Data

Wickham (2014) names five frequent ways in which real datasets break the tidy rules:

Problem	Example	Fix in pandas
Column headers are values, not names	Columns 2022, 2023, 2024 hold sales	melt
Several variables in one column	“m_25_34” mixes sex and age group	str.extract
Variables in both rows and columns	Rows tmax, tmin; columns d1 … d31	melt, then pivot
Several types of unit in one table	Song details repeated on every chart week	split, then merge
One type of unit in several tables	One sales file per month	concat

Read the first row first. It is the most common problem, and Practice 3 fixes it.

Key point: name the problem first; each of the five has a standard fix in pandas.

Objective 05

From Wide to Long with melt

Wide: one column per week

student	programme	quiz_w1	quiz_w2	quiz_w3
S01	MSc DS	8	7	9.0
S02	MSc DS	6	5	NaN
S03	MBA	9	10	9.0
S04	MSc CS	7	8	6.0

melt

Long (tidy): one row per student-week

student	programme	week	score
S01	MSc DS	1	8.0
S01	MSc DS	2	7.0
S01	MSc DS	3	9.0
S02	MSc DS	1	6.0
S02	MSc DS	2	5.0
S02	MSc DS	3	NaN

… first 6 of 12 rows, sorted by student and week

Critical point: the missing score becomes an explicit row. Decide on purpose whether to keep it or drop it.

- **melt** turns headers that are values (quiz_w1 … quiz_w3) into one variable, week.
- 4 students × 3 weeks become **12 rows**; a new week adds rows, not columns.

Objective 04

Exercise: Tidy or Messy?

FIVE TABLES

1. Columns: country, 2019, 2020, 2021, 2022 (GDP per capita).

2. Columns: student_id, course, grade (one row per student per course).

3. A column blood_pressure with values such as “120/80”.

4. Columns: date, station, pm25, pm10 (one row per station per day).

5. Twelve files, sales_jan.csv to sales_dec.csv, with identical columns.

“They all open in Excel, so they are clean, aren't they?”

YOUR TASK · PAIRS · 4 MIN

1. Decide whether each table is tidy or messy for analysis.

2. For each messy table, name Wickham's problem.

3. Say which pandas function would fix it.

Check Your Understanding

A table has the columns country, 2019, 2020 and 2021, with GDP per capita in each cell. Which statement is correct?

A It is tidy, because each country appears in exactly one row.

B It is messy: the headers are values of a year variable, so melt it.

C It is messy, because it should be stored as Parquet, not CSV.

D It is tidy, because it has no missing values and no merged cells.

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Columnar format

A file that stores each column together, such as Parquet.

Character encoding

The mapping from bytes to characters, such as UTF-8.

Data dictionary

A table that documents each column: type, values, meaning.

Tidy data

Each variable a column, each observation a row, each unit a table.

Schema

The names and data types of a dataset's fields.

Metadata

Data that provides information about other data.

Data catalogue

A searchable inventory of an organisation's datasets.

melt

The pandas method that turns columns into rows (wide to long).

Summary

In this lesson, you learned how to:

- **O1** Compare text and binary, row-based and columnar file formats
- **O2** Identify encoding, delimiter, decimal, date and missing-value pitfalls
- **O3** Distinguish technical, process and business metadata; write a data dictionary
- **O4** Explain the three tidy data principles and the common messy-data problems
- **O5** Apply melt to reshape a wide table into tidy, long form

Key takeaway: read the format correctly, document the meaning, and give the table the shape the question needs.

Next lesson

3-3 Acquiring Data with Python

DS 501 · WEEK 3 · LESSON

3-3

Acquiring Data with Python

Lesson 3 of 3

In this lesson

01 Reading files with pandas, from a path or a URL

02 How the web delivers data: requests, responses, status codes

03 Calling a REST API and flattening nested JSON

04 Scraping an HTML table, and acquiring data reproducibly

45 minutes

Builds on lessons 3-1 and 3-2

Objectives

By the end of this lesson, you will be able to:

01 **Apply** pandas readers to load files and check what was loaded

Checked in: Quiz 3 · question 11

02 **Explain** HTTP requests, query parameters and status codes

Checked in: Quiz 3 · question 12

03 **Apply** requests and pd.json_normalize to get API data as a table

Checked in: Quiz 3 · question 13 and Practice 3

04 **Apply** read_html and BeautifulSoup to extract an HTML table

Checked in: Quiz 3 · question 14 and Practice 3

05 **Evaluate** whether acquisition is reproducible, polite and secure

Checked in: Quiz 3 · question 15

Look for the objective chip in the top-right corner of each slide: it shows which goal that slide serves.

Objective 05

Case: Downloading by Hand

THE SITUATION

An analyst at a regional development agency prepares a weekly report on weather and the economy.

Her manager now wants the same report for each of North Macedonia's eight statistical regions.

> “Every Monday I copy the weather from one website and GDP figures from another into Excel. It takes an hour, and last week I pasted the wrong year.”

YOUR TASK · PAIRS · 5 MIN

1. What are the risks of this manual process?
2. Which steps could a script do, and with which tool?
3. What should the script record so that the numbers can be checked later?

Objective 01

Reading Files with pandas

Each reader takes a local path or a URL and returns a DataFrame.

Format	Function	Useful parameters
CSV / TSV	pd.read_csv	sep, decimal, encoding, na_values, parse_dates
Excel	pd.read_excel	sheet_name, skiprows, usecols
JSON / JSON lines	pd.read_json	orient; lines=True for one record per line
Parquet	pd.read_parquet	columns: read only what you need
SQL query	pd.read_sql	the query text and a database connection
HTML tables	pd.read_html	match; returns a list of tables

In practice

After loading, always check **shape**, **dtypes**, **isna().sum()** and a few rows.

Note: read_html returns a list, one DataFrame per table on the page.

Reading a Difficult CSV File

```python
# Python · an illustrative European export
df = pd.read_csv("sales_2025.csv",
        sep=";", decimal=",", thousands=".",
        encoding="utf-8",
        parse_dates=["date"], dayfirst=True,
        na_values=["-", "n/a"])
df.dtypes
```

1 The file name is illustrative: a typical export from accounting software.

← **2 · Critical:** without decimal="," the amounts stay text, and any sum fails or goes wrong.

3 Read Cyrillic and ë, ç, ş correctly; try cp1251 for old Windows files.

4 Read 03/04/2025 as 3 April, not 4 March.

5–6 Turn missing codes into NaN, then confirm the amounts are numeric.

⚠ **Common mistake:** fixing the file by hand in Excel. Fix it in code, so the same step works on next month's file.

Objective 02

How the Web Delivers Data: HTTP

Web pages and APIs use HTTP: a client sends a **request**, and the server returns a **response**.

Request

Method: GET reads data; POST sends data, such as a form.

URL: the endpoint, then parameters after ? joined by &.

Headers: User-Agent, accepted format, sometimes an API key.

Response

Status code: did the request succeed, and if not, why?

Headers: e.g. Content-Type: application/json.

Body: HTML, JSON, XML, CSV or a binary file.

Key point: a **REST API** exposes data as resources at URLs and follows these conventions; a browser and requests are both HTTP clients

Objective 02

HTTP Status Codes You Will Meet

Code	Meaning	What to do
200	OK: the request succeeded	Parse the body
301 / 302	Moved to another URL	requests follows it automatically
400	Bad request: a parameter is wrong	Check names and values
401 / 403	Unauthorised / forbidden	Check the API key or permissions
404	Not found	Check the URL and the identifier
429	Too many requests	Wait, then retry more slowly
500 / 503	Server error / unavailable	Retry later; not your fault

Critical point: r.raise_for_status() stops the script on any 4xx or 5xx code, instead of parsing an error page.

- 2xx means success, 4xx a problem with your request, 5xx a problem on the server.

Objective 03

Calling a REST API with requests

```
# URL = Open-Meteo archive endpoint (Practice 3)
params = {"latitude": 41.9981, "longitude": 21.4254,
    "start_date": "2024-01-01", "end_date": "2024-12-31",
    "daily": "temperature_2m_mean,precipitation_sum",
    "timezone": "Europe/Skopje"}
r = requests.get(URL, params=params, timeout=20)
r.raise_for_status()
```

1 Skopje's coordinates choose the place.

← **2 · Critical:** a fixed date range makes the request repeatable; “the last 30 days” is not.

3–4 Two daily variables, in the local time zone.

5–6 Send the GET request with a timeout; stop on any 4xx or 5xx status.

In practice

Open-Meteo needs no key for non-commercial use. Most commercial APIs need a key and set stricter limits.

Objective 03

Flattening Nested JSON: World Bank API

```
[
  {"page": 1, "total": …},
  [
    {"indicator": {"id": "NY.GDP.PCAP.CD", …},
     "country": {"id": "MK",
                 "value": "North Macedonia"},
     "countryiso3code": "MKD",
     "date": "2010", "value": …},
    …
  ]
]
```

Critical point: element [0] is paging metadata; the records are in element [1].

json_ normalize

country.value	date	value
North Macedonia	2010	…
North Macedonia	2011	…
…	…	…

- The World Bank API returns **[metadata, records]**; each record nests objects inside it.
- **pd.json_normalize** turns nested objects into dotted columns such as country.value.

Extracting a Table: BeautifulSoup and read_html

```
# html = the small page from Practice 3, section 4
from bs4 import BeautifulSoup
soup = BeautifulSoup(html, "html.parser")
rows = [[td.get_text(strip=True) for td in tr.find_all(["td", "th"])]
        for tr in soup.select("#lines tr")]
print(rows[:2])
lines = pd.read_html(io.StringIO(html))[0]
```

1–2 Parse the page into a tree of tags.

← **3–4 · Critical:** select the table by its id, then take the text of every cell in every row.

5 The header row, then line 2: 14.2 km, 6 buses per hour.

6 The short way: read every table, take the first.

⚠ **Common mistake:** calculating with scraped values before converting them: every value starts as **text**.

Reproducible Data Acquisition

Acquisition is the first step of the pipeline: if it cannot be rerun, nothing after it can be checked.

01

Write acquisition as code in a notebook or script, never as manual downloads.

02

Keep raw data unchanged in a data/raw folder; write cleaned files elsewhere.

03

Record the source: URL, parameters, date of access and licence.

04

Fix the request: a fixed date range, not “the last 30 days”.

05

Cache the raw response in a file, and read the file on later runs.

06

Fail loudly: catch errors and say which source was actually used.

In practice

Practice 3 falls back to a clearly labelled **synthetic** sample when an API is unreachable, and prints which source it used.

Being a Good API Citizen

Keep keys secret

Store API keys in an environment variable or Colab secret, never in the notebook or on GitHub.

Respect limits

Providers limit requests per minute or day, and send large results in pages: request them one by one.

Time out and retry

Set a timeout. After 429 or 5xx, wait and retry with a growing delay.

Follow the terms

Read the licence: who may use the data, for what purpose, and how to cite it.

Common mistake: pushing a notebook with an API key to a public repository. Automated bots scan public code for keys.

Objective 05

If you cannot rerun your data acquisition, nobody can check what follows.

Every chart, model and conclusion depends on the data you collected. Code, recorded sources and unchanged raw files let anyone, including you next month, rebuild it exactly.

Rule of thumb: a colleague should be able to rebuild your raw data from your notebook alone.

Objective 01

Exercise: Which Tool?

FIVE TASKS

1. Daily exchange rates from a central bank service that returns JSON.
2. A table of city populations on a Wikipedia page.
3. Last month's orders in the company's PostgreSQL database.
4. An open-data file published as a semicolon-separated CSV.
5. Repository statistics from the GitHub REST API.

"Can you automate all five by next Monday?"

YOUR TASK · PAIRS · 5 MIN

1. Choose a tool for each: read_csv, requests + JSON, read_html / BeautifulSoup, or read_sql.
2. Which parameters or settings will each one need?
3. Which task should start with a check of the terms of use?

Check Your Understanding

A script that downloads data from an API in a loop suddenly receives status code 429 on every request. What should it do?

A Parse the response body: a 4xx code still contains the data.

B Check the API key, because the request was not authorised.

C Wait, then retry more slowly, with a growing delay between attempts.

D Report a bug to the provider, because the server has failed.

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

HTTP request

A message asking a server for a resource: method, URL and headers.

Status code

A three-digit number in a response that reports success or failure.

pd.json_normalize

The pandas function that flattens nested JSON into columns.

Rate limit

The maximum number of requests a provider allows in a period.

Query parameter

A name=value pair after ? in a URL that selects the data.

REST API

A web API that exposes data as resources at URLs, usually as JSON.

BeautifulSoup

A Python library that parses HTML so tags and text can be found.

Caching

Saving a response to a file so later runs read the file, not the API.

Summary

In this lesson, you learned how to:

- **O1** Apply pandas readers to load files and check what was loaded
- **O2** Explain HTTP requests, query parameters and status codes
- **O3** Apply requests and pd.json_normalize to get API data as a table
- **O4** Apply read_html and BeautifulSoup to extract an HTML table
- **O5** Evaluate whether acquisition is reproducible, polite and secure

Key takeaway: acquire data with code, check every response, and record where each number came from.

Next

Practice 3 · Acquiring Data

Practice 3: Acquiring Data

You collect data in four ways (files, two APIs and a web page), make it tidy and document it.

Goal

Acquire data with code from files, REST APIs and HTML, then reshape and document it.

Data

tips (**244** rows), Skopje weather 2024, World Bank GDP per capita, a bus-line page, a quiz table.

Notebook

The week 3 lab notebook, in Google Colab. No installation is needed.

Time

About 60 minutes in class; finish the exercises at home before the week 4 session.

Parts A–C

Files and Open-Meteo (§1–2), World Bank JSON (§3), scraping an HTML table (§4).

Sections 5–6

Melt a wide table into tidy form, and write a data dictionary for the weather data.

⚠ If an API is unreachable, the notebook uses a labelled **synthetic** sample. Say which source you used.

Files: A Round Trip Through Excel and JSON Lines

```
# tips = pd.read_csv(… tips.csv) → (244, 7)
tips.to_excel("tips.xlsx", index=False)
tips.to_json("tips.jsonl", orient="records", lines=True)
a = pd.read_excel("tips.xlsx")
b = pd.read_json("tips.jsonl", lines=True)
print(a.shape, b.shape, a.shape == b.shape == tips.shape)
# (244, 7) (244, 7) True
```

1–2 Write the same table as Excel and as JSON lines.

3–4 Read both files back with the matching readers.

5–6 · Measure: record the shapes; True means no rows or columns were lost.

In practice

Section 2 then requests Skopje weather from Open-Meteo and resamples it to monthly values with **resample("MS")**.

An API with Nested JSON: World Bank GDP

```
# wb_json = r.json() from the World Bank API (§3)
1  gdp = pd.json_normalize(wb_json[1])[["country.value", "date", "value"]]
2  gdp = gdp.rename(columns={"country.value": "country",
3                            "value": "gdp_pc_usd"})
4  gdp["date"] = gdp["date"].astype(int)
5  gdp = gdp.dropna().sort_values(["country", "date"])
```

← **1 · Critical:** the records sit in element [1]; json_normalize flattens their nested fields.

2–3 Give the columns short, clear names.

4 The year arrives as text; make it an integer.

5 Drop missing values and sort for plotting.

⚠ **Common mistake:** reporting the plotted values as real GDP when the notebook prints **SYNTHETIC fallback** as the source.

Scraping: From HTML Text to Numbers

```
# rows = the BeautifulSoup list from §4 (fictional lines)
t = pd.DataFrame(rows[1:], columns=rows[0])
t["Length (km)"] = t["Length (km)"].astype(float)
t["Buses per hour"] = t["Buses per hour"].astype(int)
t["km_per_bus"] = (t["Length (km)"] / t["Buses per hour"]).round(2)
t.sort_values("km_per_bus", ascending=False)
```

1 The first scraped row becomes the header.

2–3 Convert the text to numbers before calculating.

4 Length divided by buses per hour.

← **5 · Result:** line 57, Center to Saraj, is first with **9.45**; line 5 is last with 1.48.

📖 Sections 5–6: melt the quiz table into **12** tidy rows and write a data dictionary for the weather table.

Deliverables

What to Submit

Submit the completed notebook (.ipynb) on the course page. It must run from top to bottom without errors.

Part	What is marked	Points
Run	The notebook runs from top to bottom; each source line is reported	2
A	Round-trip shapes; monthly weather table and plot; parameter question	2
B	World Bank data flattened, typed and plotted with the source label	2
C	Typed table, km_per_bus result; limits of scraping explained	2
5–6	Tidy (long) quiz table with mean scores; weather data dictionary	1
Project	Your project's data sources, formats and access methods	1
	Total	10

⚠ **Common mistake:** presenting **synthetic** fallback values as real weather or GDP figures.

← **Due:** [date], before the week 4 session.

WEEK 3 · PAPERS TO READ

Papers to Read

Two papers on preparing data for use: how to structure a dataset so that it is easy to analyse, and how to document it so that others can use it responsibly.

Required

Required Reading for Week 4

Read the listed sections before the week 4 class. One question in Quiz 4 may refer to these papers.

Tidy Data

Wickham · Journal of Statistical Software 59(10), 2014

The three rules and five messy-data problems. Read sections 1–3.

Datasheets for Datasets

Gebru et al. · Communications of the ACM 64(12), 2021

Standard questions every dataset should answer. Read it all; it is short.

Bring a half-page note per required paper: the problem, the key idea, and its link to this week.

Optional

Optional Reading

Not examined. Each one deepens a theme of this week and helps with your project.

The FAIR Guiding Principles for Scientific Data Management and Stewardship
Wilkinson et al. · Scientific Data 3, 2016

Findable, accessible, interoperable, reusable. Read the principles box.

Data Organization in Spreadsheets
Broman & Woo · The American Statistician 72(1), 2018

Practical rules for spreadsheet data. Read the rules and the summary.

Python for Data Analysis, 3rd ed., chapter 6: Data Loading, Storage and File Formats
McKinney · O'Reilly, 2022 · course companion text

Every pandas reader and writer used this week, with examples.

Optional papers are not examined. Choose one that relates to your project.

Discussion

Reading Questions for Discussion

- Which of Wickham's five problems do you expect in your project data, and how will you fix each one?
- Choose a dataset you have used in this course. Which datasheet questions can its documentation answer, and which not?
- Wickham writes about structure, Gebru and colleagues about documentation. How do the two ideas complement each other?
- Which rules of tidy data did the spreadsheet from Finance (lesson 3-2) break?

Prepare for class

- Read the assigned sections
- Write a half-page note per paper
- Bring one answer to each question
- Discuss in groups of three

Key point: structure makes data easy to **analyse**; documentation makes it safe to **reuse**.

DS 501 · WEEK 3

This Week's Work

Practice 3 notebook

Finish the week 3 lab notebook, run it from top to bottom, state which sources it used, and submit the .ipynb file.

Due: [date]

Quiz 3

15 multiple-choice questions on lessons 3-1 to 3-3, and one short applied question on evaluating a data source.

Due: [date]

Project milestone 1: topic and question

Your decision, your question and its unit of analysis, and at least one data source checked with the seven criteria.

Due: [date]

Next week: Databases and SQL: how relational databases store an organisation's data, and how to query it with SQL from Python.